

Batch File (.bat or .cmd)

- A batch file is a text file that contains a series of commands to be executed by the command-line interpreter in operating systems like Windows or DOS. These commands are run in sequence, one after the other, making it possible to automate repetitive tasks.
- Batch files typically have a `.bat` or `.cmd` file extension and can include commands to execute programs, copy files, create directories, delete files, and more. They are often used for system administration, automation, and simplifying tasks that would otherwise require manual entry of commands.
- DOS commands are case-insensitive. 大小寫不分

Common Uses

- Automating backups
- Running multiple programs in sequence
- Deleting or moving files
- Scheduling tasks through Task Scheduler
- Customizing the startup environment

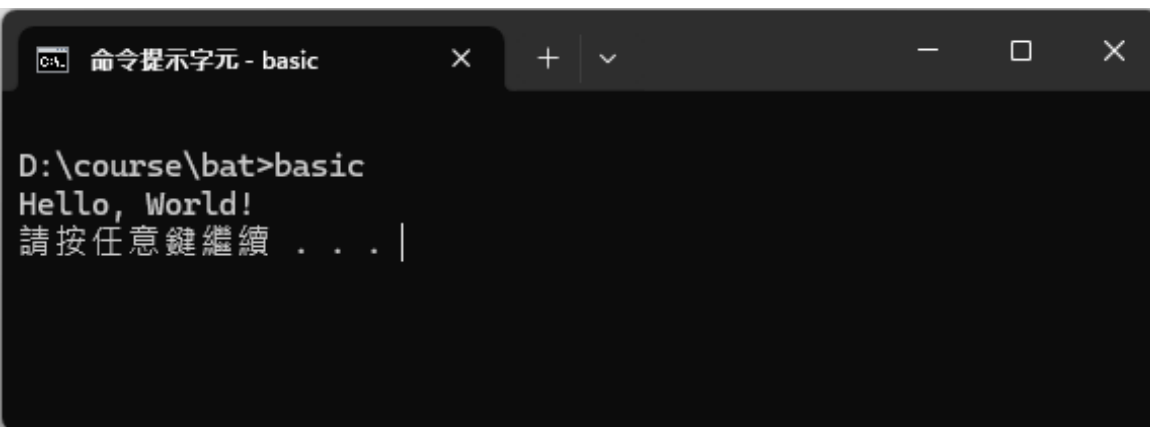
Steps needed to run a batch script

- Open a Text Editor such as Notepad++ (or PSPad)
- Write the script (basic Commands) and save file with a `.bat` (or `.cmd`) extension
- Open the command prompt (`cmd.exe`).
- Navigate to the directory where the `.bat` file is stored.
- Type the name of the file and press Enter to run the batch file.

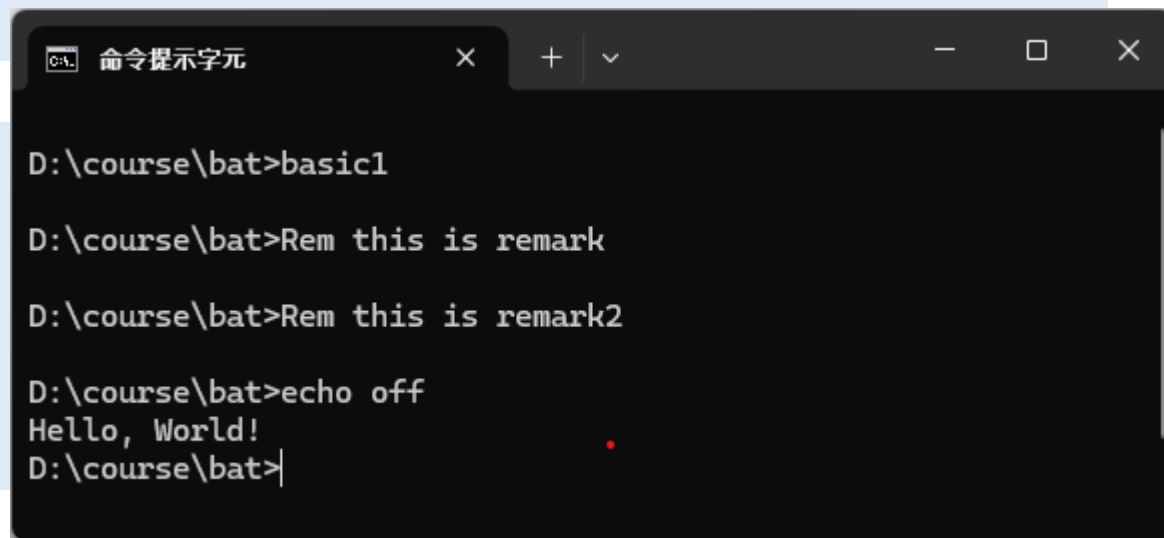
Basic structure

```
@echo off
Rem this is remark
:: this is comments
echo Hello, World!
pause
```

```
Rem this is remark
Rem this is remark2
:: this is comments
echo off
echo Hello, World!
```



```
命令提示字元 - basic
D:\course\bat>basic
Hello, World!
請按任意鍵繼續 . . . |
```



```
命令提示字元
D:\course\bat>basic1
D:\course\bat>Rem this is remark
D:\course\bat>Rem this is remark2
D:\course\bat>echo off
Hello, World!
D:\course\bat>
```

- By default, a batch file will display its command as it runs. The purpose of this first command is to turn off this display.
- The command "echo off" turns off the display for the whole script, except for the "echo off" command itself.
- The "at" sign "@" in front makes the command apply to itself as well.

Batch script - commands

Command	Description
help	show the list of commands
dir	list the contents of a directory
help dir	show the help of the specified command: dir
dir /?	show the usage of dir command

```
C:\Users\user>help
要了解特定命令的詳細資訊，請輸入 HELP 命令名稱
ASSOC          顯示或修改檔案副檔名關聯。
ATTRIB         顯示或變更檔案屬性。
BREAK          設定或清除擴充的 CTRL+C 檢查。
BCDEDIT        設定 boot 資料庫檔案的內容來控制開機載入。
CACLS          顯示或修改檔案的存取控制清單 (ACLs)。
CALL           從一個批次程式呼叫另一個批次程式。
CD             顯示或變更目前目錄的名稱。
CHCP           顯示或設定作用中的字碼頁編號。
CHDIR          顯示或變更目前目錄的名稱。
CHKDSK         檢查磁碟並顯示狀態報告。
CHKNTFS        顯示或修改開機時的磁碟檢查。
```

```
C:\Users\user>help dir
顯示檔案清單以列出目錄中的檔案及子目錄。

DIR [drive:][path][filename] [/A[[:]attributes]] [/B] [/C] [/D] [/L] [/N]
    [/O[[:]sortorder]] [/P] [/Q] [/R] [/S] [/T[[:]timefield]] [/W] [/X] [/4]

[drive:][path][filename]
    指定要顯示的磁碟機、目錄或檔案。

/A          顯示具有指定之屬性的檔案。
attributes D 目錄                      R 唯讀檔案
            H 隱藏檔案                  A 待封存檔案
            S 系統檔案                  I 非內容索引檔案
            L 重新分析點                O 離線檔案
            - 首碼表示否定

/B          使用單純格式 (沒有標頭資訊或摘要)。
/C          在檔案大小中顯示千位分隔符號。這是預設值。使用 /-C 可停用
            分隔符號的顯示。
/D          與寬的列表格式相同，但是依照欄來排序。
```

ASSOC	顯示或修改檔案副檔名關聯。	DISKPART	顯示或設定磁碟分割內容。
ATTRIB	顯示或變更檔案屬性。	DOSKEY	編輯命令列、重新呼叫 Windows 命令和建立巨集。
BREAK	設定或清除擴充的 CTRL+C 檢查。	DRIVERQUERY	顯示目前裝置驅動程式狀態和內容。
BCDEDIT	設定 boot 資料庫檔案的內容來控制開機載入。	ECHO	顯示訊息、開啟或關閉命令回音。
CACLS	顯示或修改檔案的存取控制清單 (ACLs)。	ENDLOCAL	結束批次檔環境變更的當地語系化工作。
CALL	從一個批次程式呼叫另一個批次程式。	ERASE	刪除一或多個檔案。
CD	顯示或變更目前目錄的名稱。	EXIT	結束 CMD.EXE 程式 (命令直譯器)。
CHCP	顯示或設定作用中的字碼頁編號。	FC	比較兩個或兩組檔案，然後顯示兩者之間的相異處。
CHDIR	顯示或變更目前目錄的名稱。	FIND	搜尋一或多個檔案中的文字字串。
CHKDSK	檢查磁碟並顯示狀態報告。	FINDSTR	在檔案中搜尋字串。
CHKNTFS	顯示或修改開機時的磁碟檢查。	FOR	在一組檔案中的每個檔案執行一個特定的命令。
CLS	清除螢幕。	FORMAT	將磁碟格式化供 Windows 使用。
CMD	開始新的 Windows 命令直譯器。	FSUTIL	顯示或設定檔案系統內容。
COLOR	設定預設主控台的前景和背景色彩。	FTYPE	顯示或修改用於檔案副檔名關聯中的檔案類型。
COMP	比較兩個或兩組檔案的內容。	GOTO	將 Windows 命令直譯器指向批次程式中已經加了標籤的列。
COMPACT	顯示或變更 NTFS 磁碟分割上的檔案壓縮。	GPRESULT	顯示電腦或使用者的群組原則資訊。
CONVERT	將 FAT 磁碟區轉換成 NTFS 格式。您不可轉換目前的磁碟機。	GRAFTABL	讓 Windows 在圖形模式下顯示擴充的字元集。
COPY	將一個或數個檔案複製到另一個位置。	HELP	為 Windows 命令提供說明資訊。
DATE	顯示或設定日期。	ICACLS	顯示、修改、備份或還原檔案與目錄的 ACL 。
DEL	刪除檔案。	IF	在批次程式中執行有條件的處理。
DIR	顯示目錄中的檔案和子目錄清單。	LABEL	建立、變更或刪除磁碟的磁碟區標籤。

MD	建立目錄。	SET	顯示、設定或移除 Windows 環境變數。
MKDIR	建立目錄。	SETLOCAL	開始執行批次檔中環境變更的當地語系化工作。
MKLINK	建立符號連結與永久連結	SC	顯示或設定服務 (背景處理程序)。
MODE	設定系統裝置。	SCHTASKS	排定電腦上要執行的命令與程式。
MORE	以一次一頁螢幕的方式來顯示輸出。	SHIFT	變更批次檔中可取代參數的位置。
MOVE	將一或多個檔案從某個目錄移動到另一個目錄。	SHUTDOWN	允許從本機或遠端將電腦關機。
OPENFILES	顯示遠端使用者因檔案共用而開啟的檔案。	SORT	將輸入排序。
PATH	顯示或設定可執行檔的搜尋路徑。	START	開始另一個視窗來執行指定的程式或命令。
PAUSE	暫停處理批次檔並顯示訊息。	SUBST	將路徑與磁碟機代號相關聯。
POPD	還原 PUSHD 儲存之目前目錄先前的值。	SYSTEMINFO	顯示電腦特定內容與設定。
PRINT	列印文字檔案。	TASKLIST	顯示所有目前正在執行的工作，包括服務。
PROMPT	變更 Windows 命令提示字元。	TASKKILL	刪除或停止正在執行的處理程序或應用程式。
PUSHD	儲存目前的目錄，然後變更它。	TIME	顯示或設定系統時間。
RD	移除目錄。	TITLE	設定 CMD.EXE 工作階段的視窗標題。
RECOVER	從損壞或不良的磁碟中復原可讀取的資訊。	TREE	以圖形方式顯示磁碟機或路徑的目錄結構。
REM	在批次檔或 CONFIG.SYS 中記錄註解 (備註)。	TYPE	顯示文字檔的內容。
REN	重新命名一或多個檔案。	VER	顯示 Windows 版本。
RENAME	重新命名一或多個檔案。	VERIFY	告訴 Windows 是否要檢查您的檔案是否已正確寫入到磁碟。
REPLACE	取代檔案。	VOL	顯示磁碟區標籤與序號。
RMDIR	移除目錄。	XCOPY	複製檔案與目錄樹狀結構。
ROBOCOPY	用於複製檔案與目錄樹狀結構的進階公用程式	WMIC	顯示互動式命令殼層中的 WMI 資訊。

Redirection (>, >>, <) and pipes (|)

<code>command > filename</code>	Redirect command output to a file
<code>command >&n</code>	Redirect command output to the input of handle n
<code>command >> filename</code>	APPEND into a file
<code>command < filename</code>	Type a text file and pass the text to command.
<code>command <&n</code>	Read input from handle n and write it to command.

STDIN = 0 Keyboard input

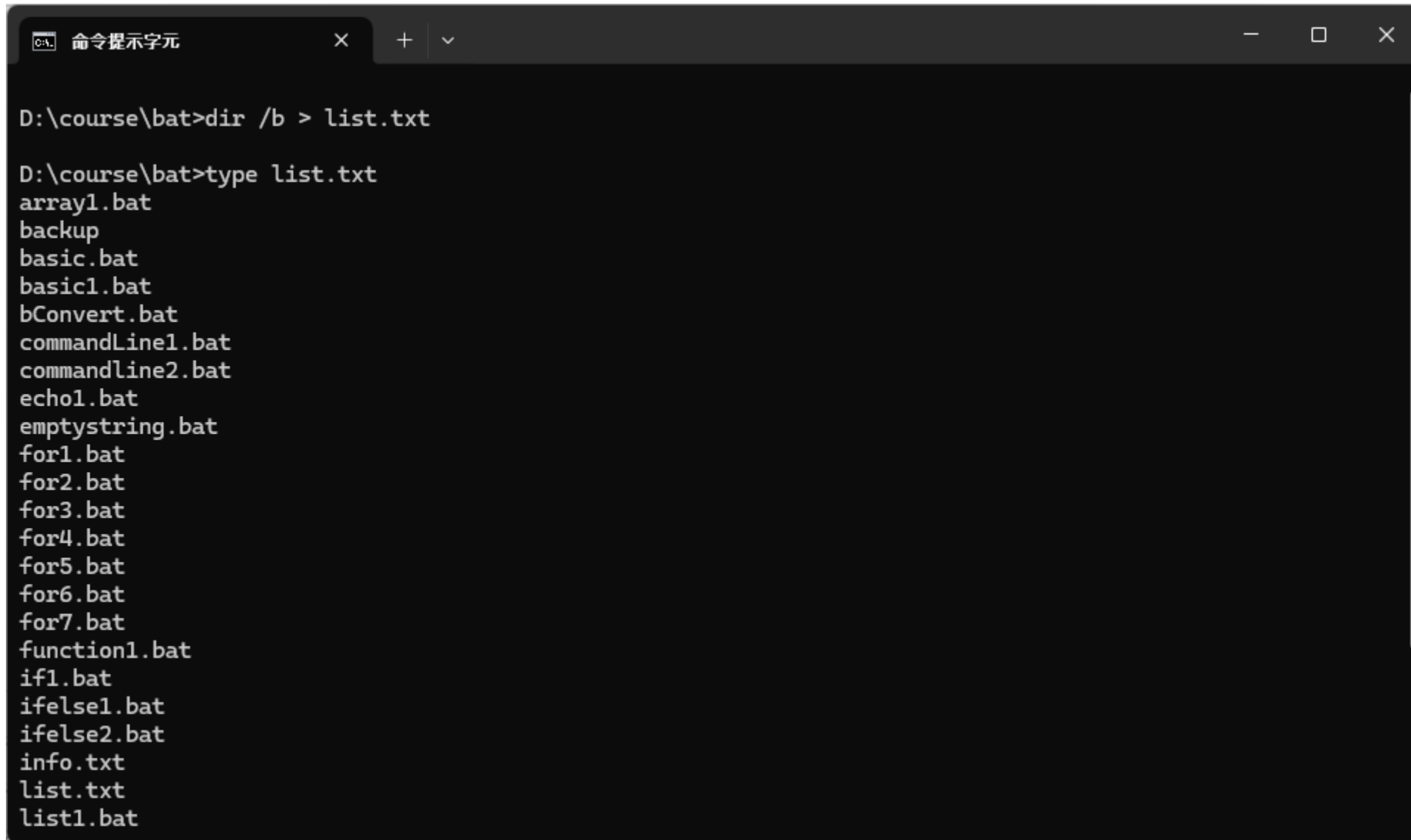
STDOUT = 1 Text output

STDERR = 2 Error text output

UNDEFINED = 3-9 (In PowerShell 3.0+ these are defined)

- When redirection is performed without specifying a numeric handle, the default < redirection input operator is zero (0) and the default > redirection output operator is one (1). This means that '>' alone will not redirect error messages.

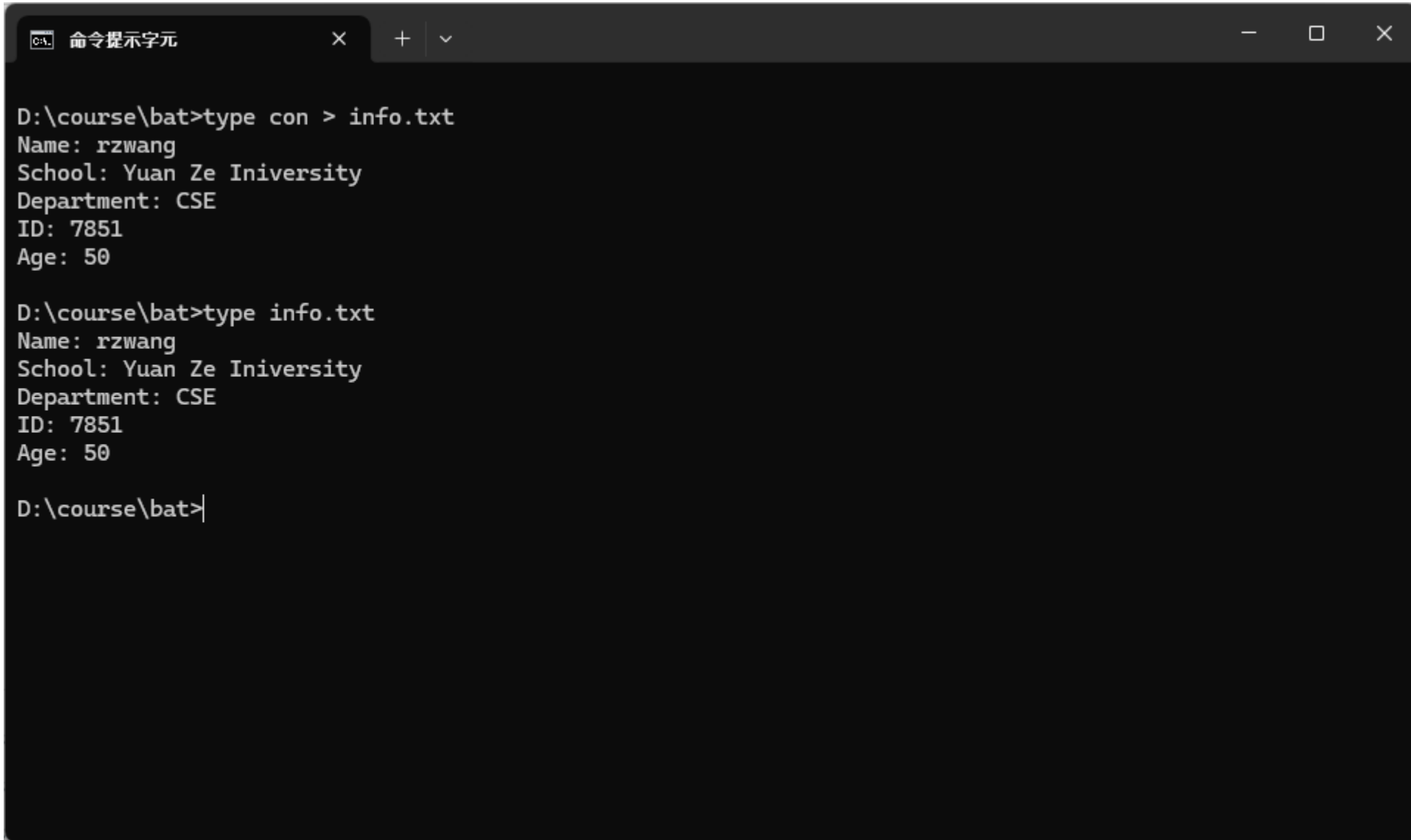
dir /b > list.txt



```
命令提示字元
D:\course\bat>dir /b > list.txt

D:\course\bat>type list.txt
array1.bat
backup
basic.bat
basic1.bat
bConvert.bat
commandLine1.bat
commandline2.bat
echo1.bat
emptystring.bat
for1.bat
for2.bat
for3.bat
for4.bat
for5.bat
for6.bat
for7.bat
function1.bat
if1.bat
ifelse1.bat
ifelse2.bat
info.txt
list.txt
list1.bat
```

type con > info.txt

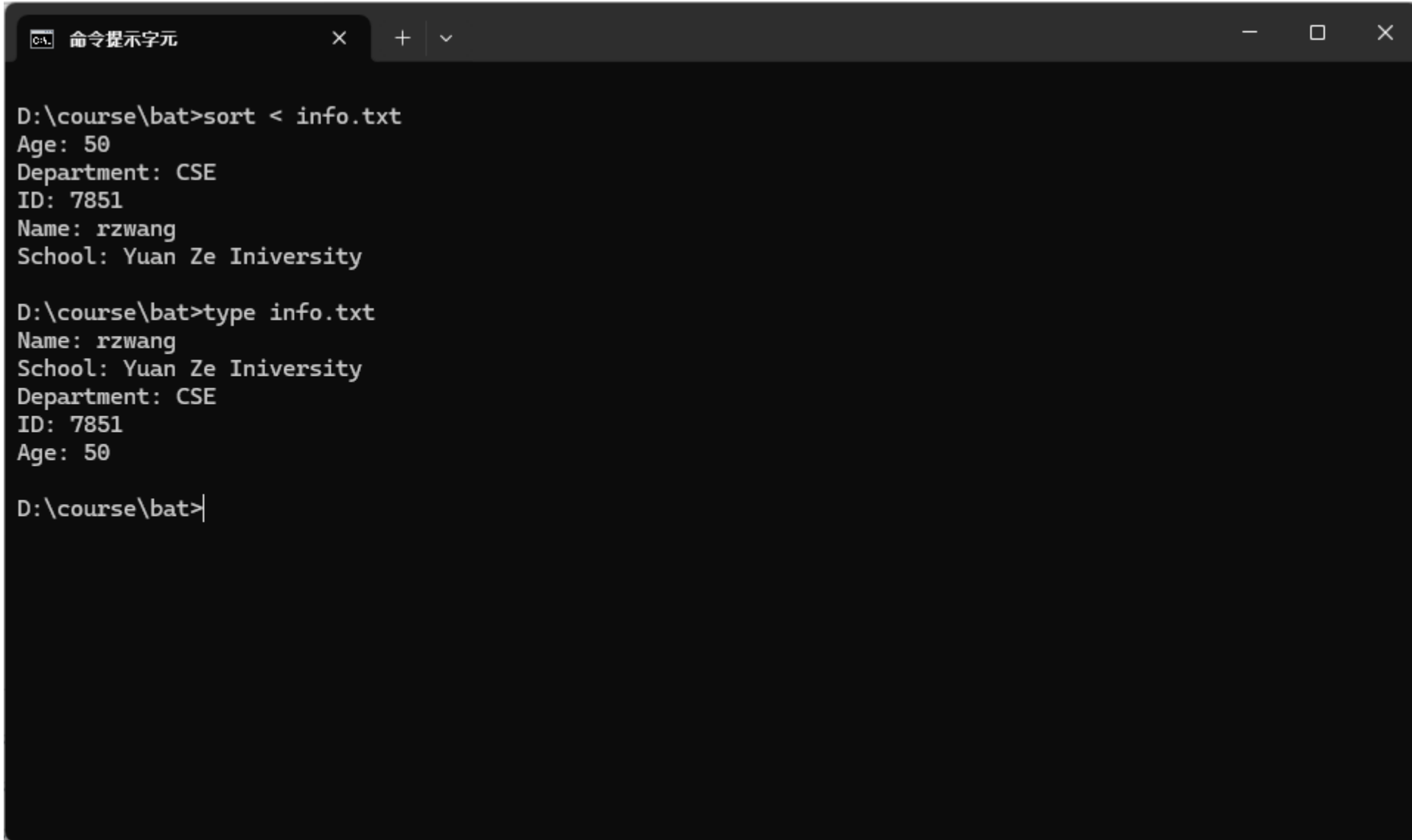


```
D:\course\bat>type con > info.txt
Name: rzwang
School: Yuan Ze Iniversity
Department: CSE
ID: 7851
Age: 50

D:\course\bat>type info.txt
Name: rzwang
School: Yuan Ze Iniversity
Department: CSE
ID: 7851
Age: 50

D:\course\bat>
```

sort < info.txt



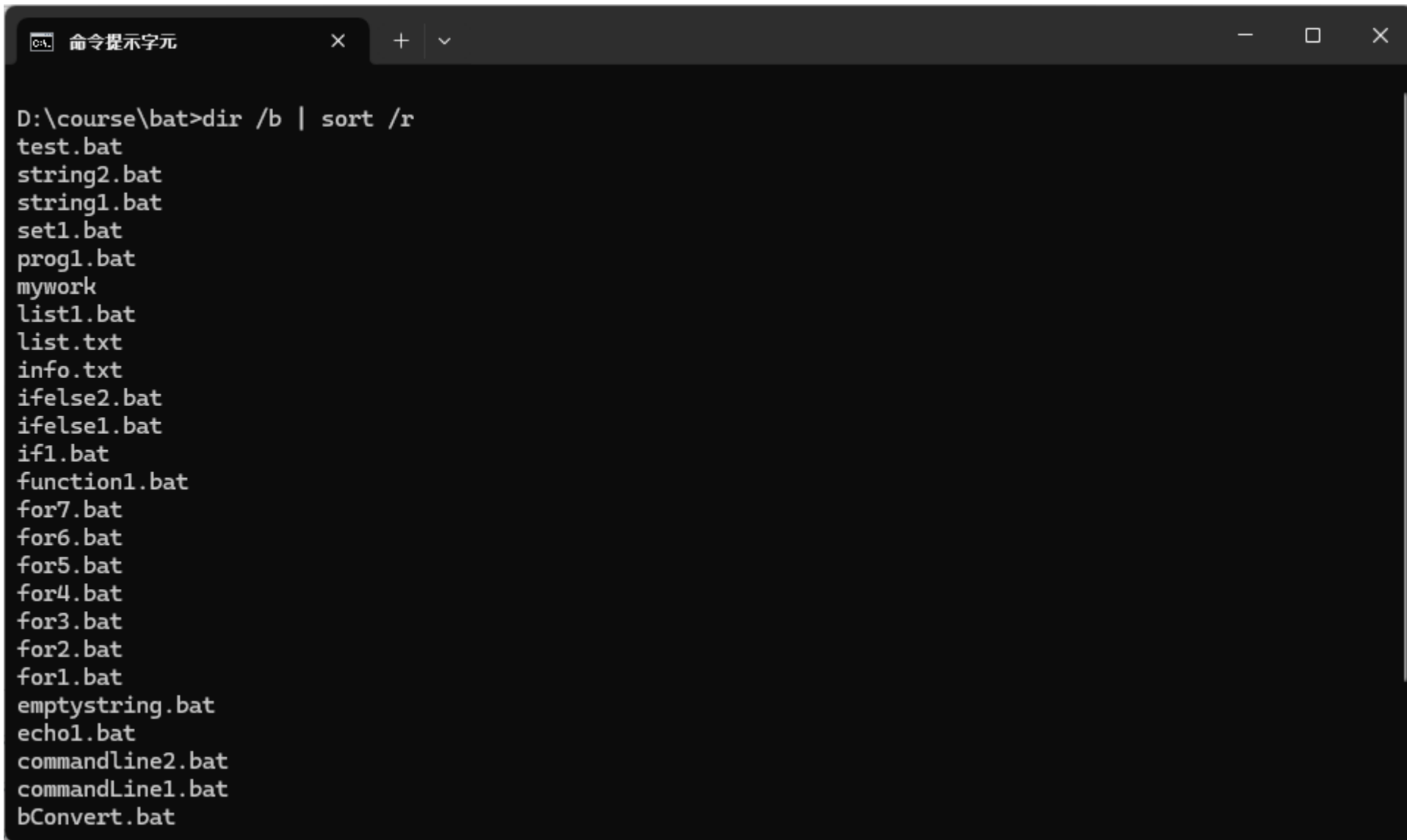
The screenshot shows a Windows Command Prompt window with the title bar '命令提示字元'. The command prompt is at the directory 'D:\course\bat'. The user has entered the command 'sort < info.txt', which has sorted the contents of 'info.txt' in ascending order. The sorted output is displayed on the screen. Below this, the user has entered 'type info.txt' to show the original, unsorted contents of the file. The output shows that the original file contained information about 'rzwang' from 'Yuan Ze Iniversity' (sic) in the 'CSE' department, with ID '7851' and age '50'. The prompt is currently at 'D:\course\bat>'.

```
D:\course\bat>sort < info.txt
Age: 50
Department: CSE
ID: 7851
Name: rzwang
School: Yuan Ze Iniversity

D:\course\bat>type info.txt
Name: rzwang
School: Yuan Ze Iniversity
Department: CSE
ID: 7851
Age: 50

D:\course\bat>
```

dir /b | sort /r

A screenshot of a Windows Command Prompt window. The title bar at the top reads "命令提示字元" (Command Prompt) and includes standard window controls (minimize, maximize, close). The command prompt shows the current directory as "D:\course\bat" and the command "dir /b | sort /r" has been executed. The output is a list of files and subdirectories sorted in reverse alphabetical order.

```
D:\course\bat>dir /b | sort /r
test.bat
string2.bat
string1.bat
set1.bat
prog1.bat
mywork
list1.bat
list.txt
info.txt
ifelse2.bat
ifelse1.bat
if1.bat
function1.bat
for7.bat
for6.bat
for5.bat
for4.bat
for3.bat
for2.bat
for1.bat
emptystring.bat
echo1.bat
commandline2.bat
commandLine1.bat
bConvert.bat
```

set

SET variable=value

- Assigns a value to a variable.

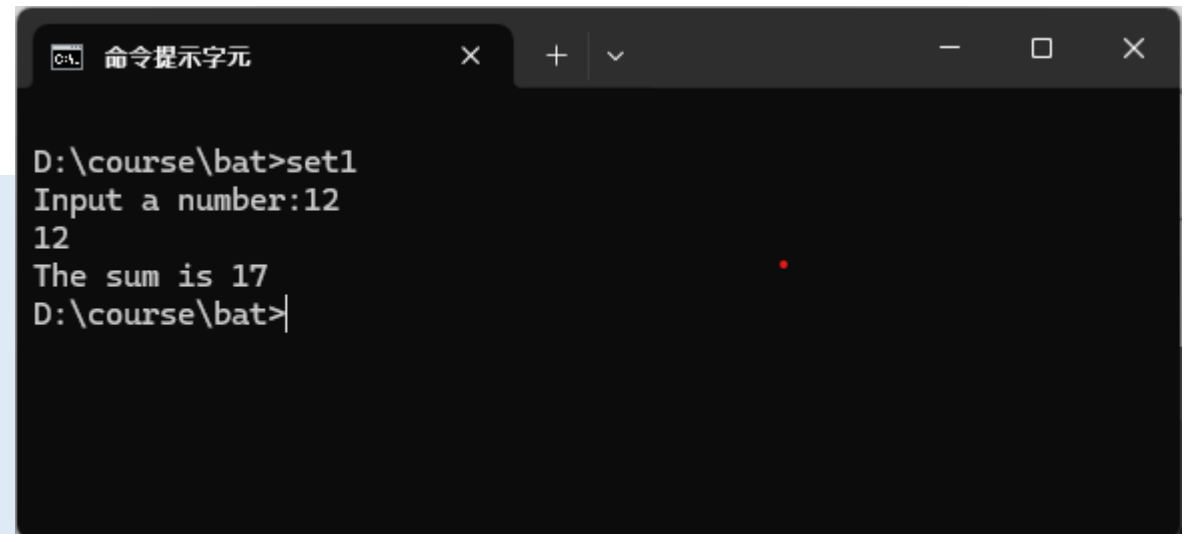
SET /A variable=expression

- Performs arithmetic operations, numeric value.

SET /P variable=promptString

- Prompts the user for input.

```
@echo off
SET /P value=Input a number:
echo %value%
SET /A sum=5 + %value%
echo The sum is %sum%
```



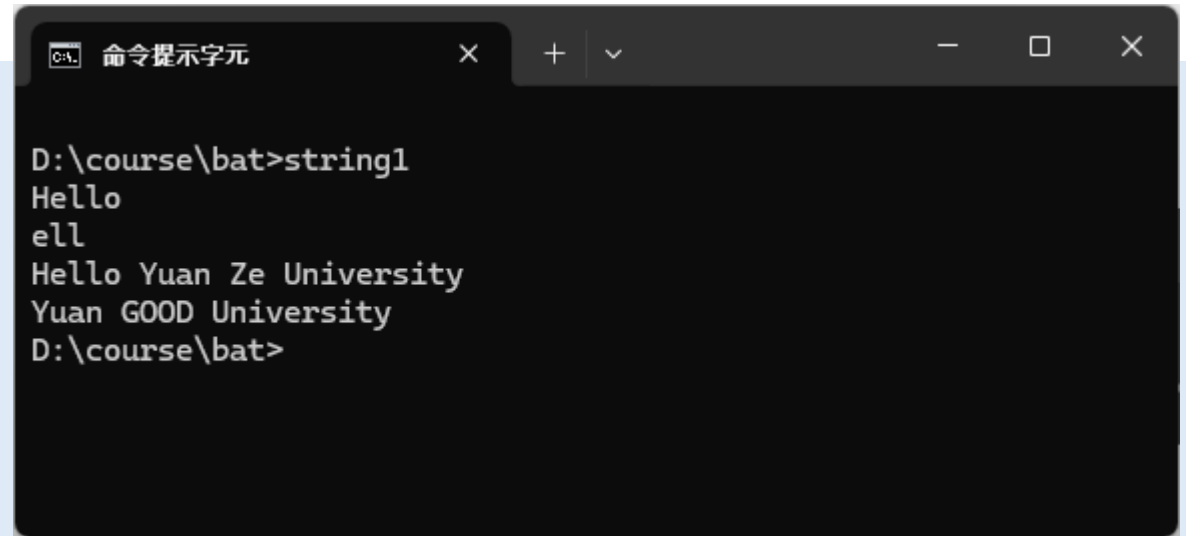
The screenshot shows a Windows Command Prompt window titled "命令提示字元" (Command Prompt). The window is dark-themed. The command prompt shows the following sequence of commands and output:

```
D:\course\bat>set1
Input a number:12
12
The sum is 17
D:\course\bat>|
```

The output demonstrates the script's functionality: it prompts for a number, reads the input '12', calculates the sum (5 + 12 = 17), and displays the result.

string

```
@echo off
SET str1=Hello
SET str2=Yuan Ze University
echo %str1:~0,5%
echo %str1:~1,-1%
echo %str1% %str2%
echo %str2:Ze=GOOD%
```

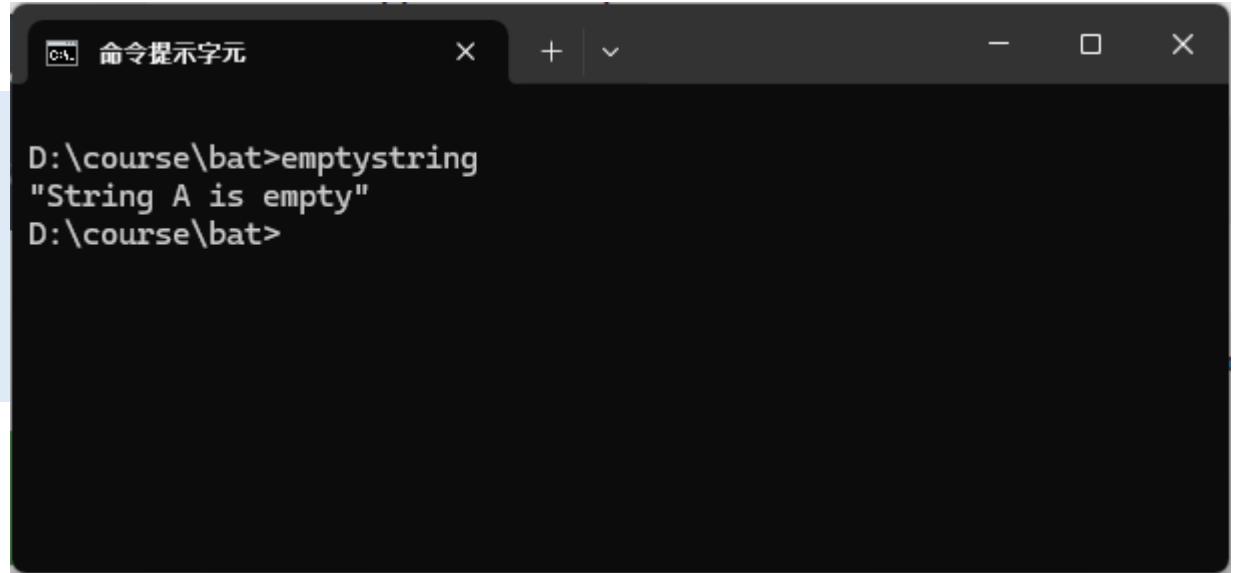


```
命令提示字元
D:\course\bat>string1
Hello
ell
Hello Yuan Ze University
Yuan GOOD University
D:\course\bat>
```

```
%variable:~num_chars_to_skip%
%variable:~num_chars_to_skip,num_chars_to_keep%
%variable:~num_chars_to_skip,-num_chars_to_keep%
%variable:~-num_chars_to_skip,num_chars_to_keep%
%variable:~-num_chars_to_skip,-num_chars_to_keep%
```

Empty string

```
@echo off
SET a=
SET b=Hello
if [%a%]==[] echo "String A is empty"
if [%b%]==[] echo "String B is empty"
```

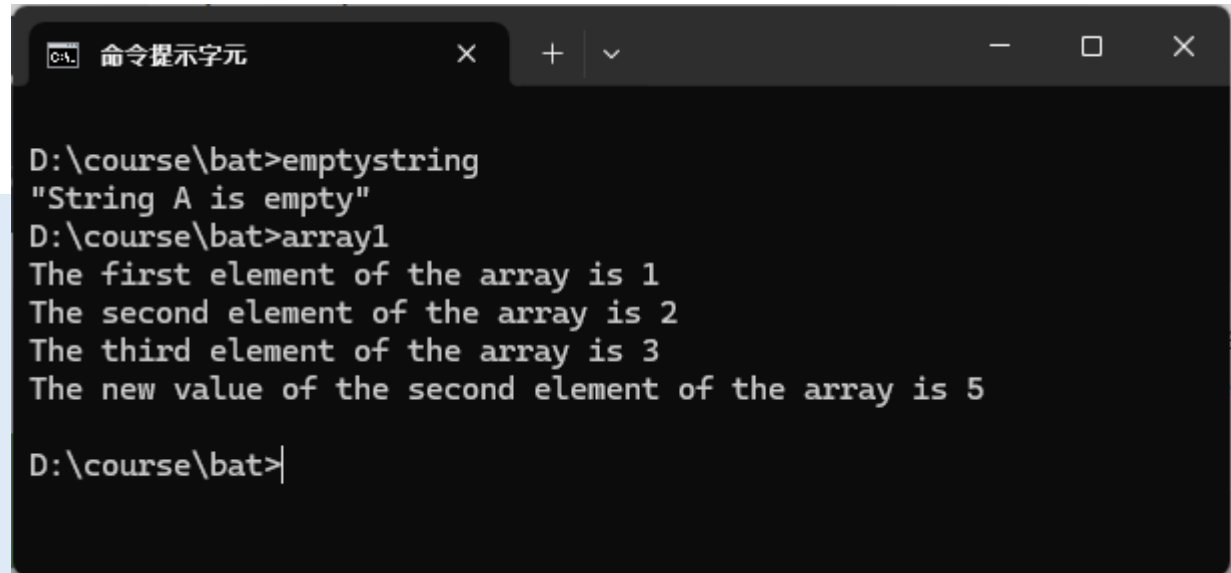


The screenshot shows a Windows Command Prompt window with the title bar '命令提示字元' (Command Prompt). The window is dark-themed. The command prompt shows the following sequence of commands and output:

```
D:\course\bat>emptystring
"String A is empty"
D:\course\bat>
```

array implementation

```
@echo off
set a[0]=1
set a[1]=2
set a[2]=3
echo The first element of the array is %a[0]%
echo The second element of the array is %a[1]%
echo The third element of the array is %a[2]%
Set a[1]=5
echo The new value of the second element of the array is %a[1]%
```

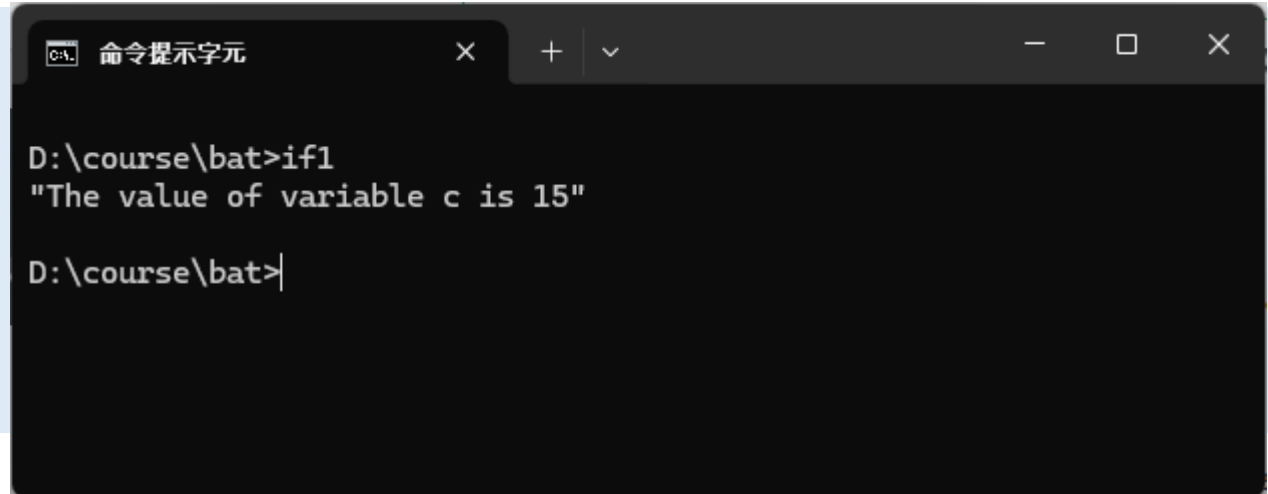
A screenshot of a Windows command prompt window titled '命令提示字元' (Command Prompt). The window shows the execution of a batch file named 'emptystring.bat' located in the directory 'D:\course\bat'. The output of the batch file is displayed as follows:

```
D:\course\bat>emptystring
"String A is empty"
D:\course\bat>array1
The first element of the array is 1
The second element of the array is 2
The third element of the array is 3
The new value of the second element of the array is 5

D:\course\bat>|
```


if

```
@echo off
SET /A a = 5
SET /A b = 10
SET /A c = %a% + %b%
if %c%==15 echo "The value of variable c is 15"
if %c%==10 echo "The value of variable c is 10"
```

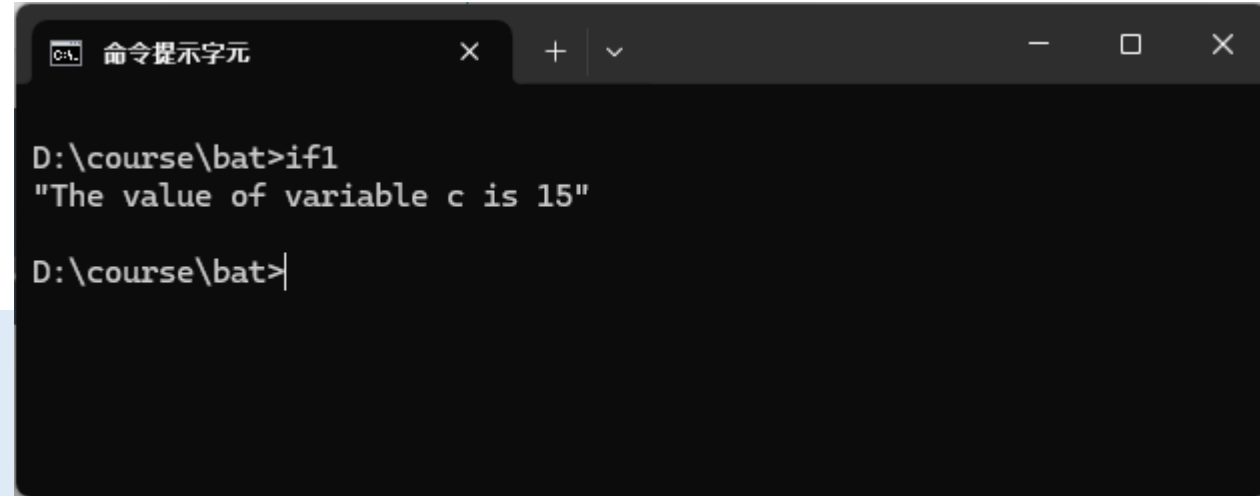
A screenshot of a Windows Command Prompt window titled "命令提示字元" (Command Prompt). The window shows the execution of a batch file located at "D:\course\bat". The command "if1" is entered, and the output is "The value of variable c is 15". The prompt then returns to "D:\course\bat>".

```
D:\course\bat>if1
"The value of variable c is 15"

D:\course\bat>
```

if else

```
@echo off
SET /A a = 5
SET /A b = 10
SET /A c = %a% + %b%
if %c%==15 (echo "The value of variable c is 15") else (echo "Unknown value")
if %c%==10 (echo "The value of variable c is 10") else (echo "Unknown value")
```

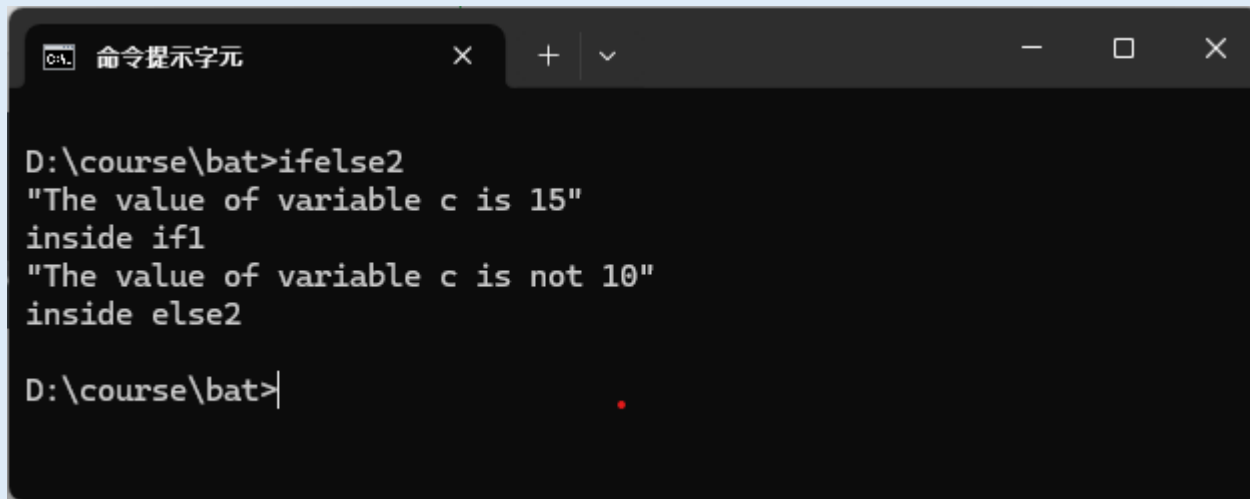
A screenshot of a Windows Command Prompt window titled "命令提示字元" (Command Prompt). The window shows the execution of a batch file located at "D:\course\bat". The first command executed is "if1", which results in the output "The value of variable c is 15". The prompt then returns to "D:\course\bat>".

```
C:\> 命令提示字元
D:\course\bat>if1
"The value of variable c is 15"
D:\course\bat>
```

if else

```
@echo off
SET /A a = 5
SET /A b = 10
SET /A c = %a% + %b%
if %c%==15 (
    echo "The value of variable c is 15"
    echo inside if1
) else (
    echo "The value of variable c is not 15"
    echo inside else1
)

if %c%==10 (
    echo "The value of variable c is 10"
    echo inside if2
) else (
    echo "The value of variable c is not 10"
    echo inside else2
)
```



```
D:\course\bat>ifelse2
"The value of variable c is 15"
inside if1
"The value of variable c is not 10"
inside else2

D:\course\bat>
```

for

In a batch file, %% is used to represent a variable in a loop (specifically, a for loop) when the script is run from within a batch file.

Here's the distinction:

- From the **command line** (when typing directly into the terminal), you would use a single % for variables.

```
for %i in (1 2 3) do echo %i
```

- In a **batch file**, you need to double the percentage signs, so you use %%:

```
for %%i in (1 2 3) do echo %%i
```

for

Options

%*: Represents all command-line arguments passed to the script.

/R: Recurses through directories.

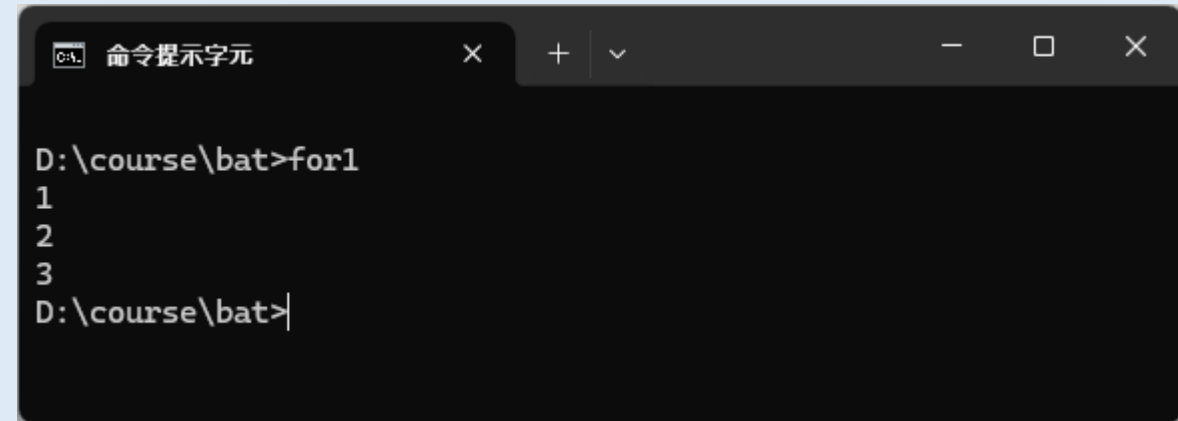
/D: Iterates over directories.

/L: Iterates over a range of numbers.

/F: Processes the output of a command or lines of a file.

for Iterating over a list of items

```
@echo off
SETLOCAL ENABLEDELAYEDEXPANSION
SET count=0
FOR %%i IN (1 2 3) DO (
    SET /A count+=1
    echo !count!
)
```



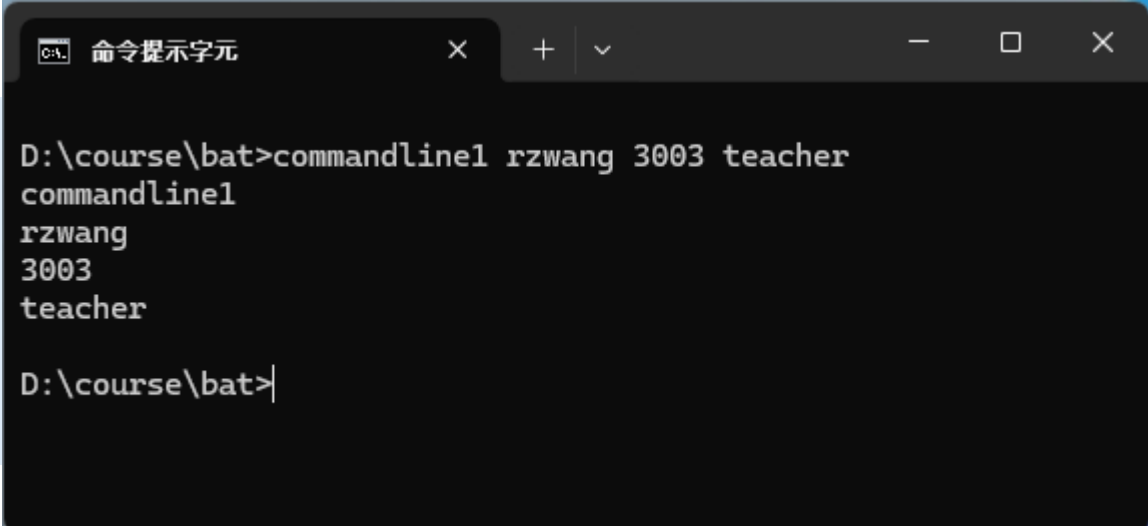
```
命令提示字元
D:\course\bat>for1
1
2
3
D:\course\bat>
```

SETLOCAL ENABLEDELAYEDEXPANSION:

- Delayed expansion allows you to modify variables within loops or conditional statements and access their updated values in real-time.
- Useful when you need to modify a variable inside a loop or block of code and reflect those changes immediately.

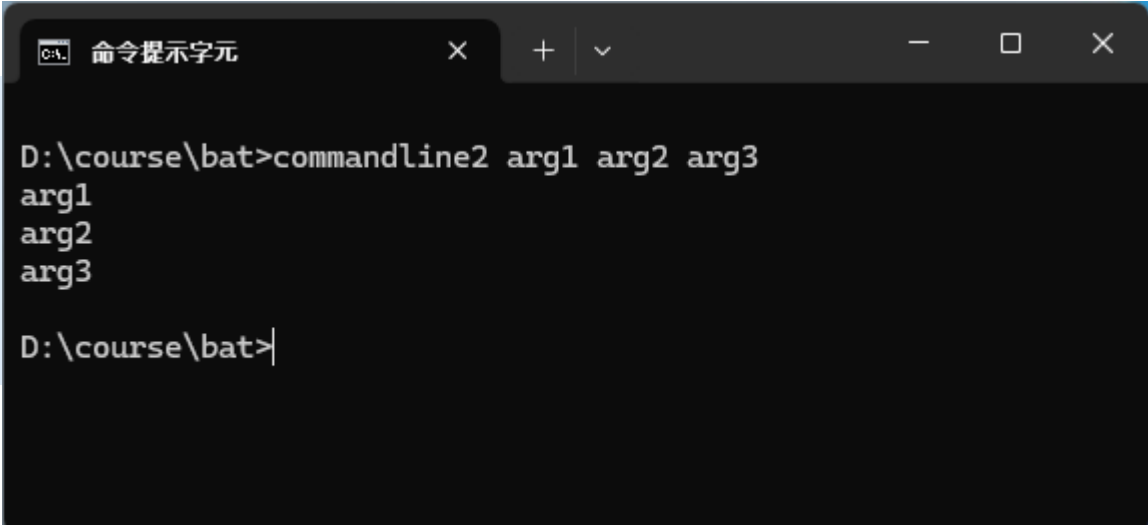
for Iterating over command-line arguments (%*)

```
@echo off
echo %0
echo %1
echo %2
echo %3
```



A screenshot of a Windows command prompt window titled "命令提示字元". The window shows the execution of a batch file named "commandline1.bat" located in the directory "D:\course\bat". The command entered is "commandline1 rzwang 3003 teacher". The output of the batch file is displayed on the next four lines: "commandline1", "rzwang", "3003", and "teacher". The prompt "D:\course\bat>" is visible at the bottom.

```
@echo off
for %%a in (%*) do (
    echo %%a
)
```

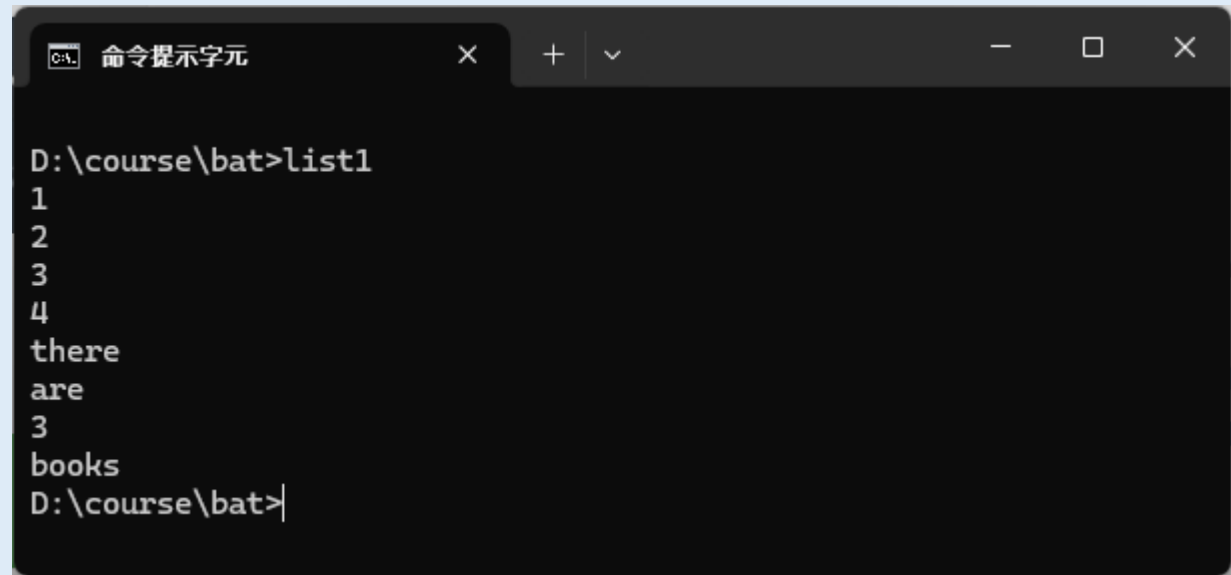


A screenshot of a Windows command prompt window titled "命令提示字元". The window shows the execution of a batch file named "commandline2.bat" located in the directory "D:\course\bat". The command entered is "commandline2 arg1 arg2 arg3". The output of the batch file is displayed on the next three lines: "arg1", "arg2", and "arg3". The prompt "D:\course\bat>" is visible at the bottom.

Using list of value

```
@echo off
set list=1 2 3 4
for %%a in (%list%) do (
    echo %%a
)

set list2=there are 3 books
for %%s in (%list2%) do (
    echo %%s
)
```

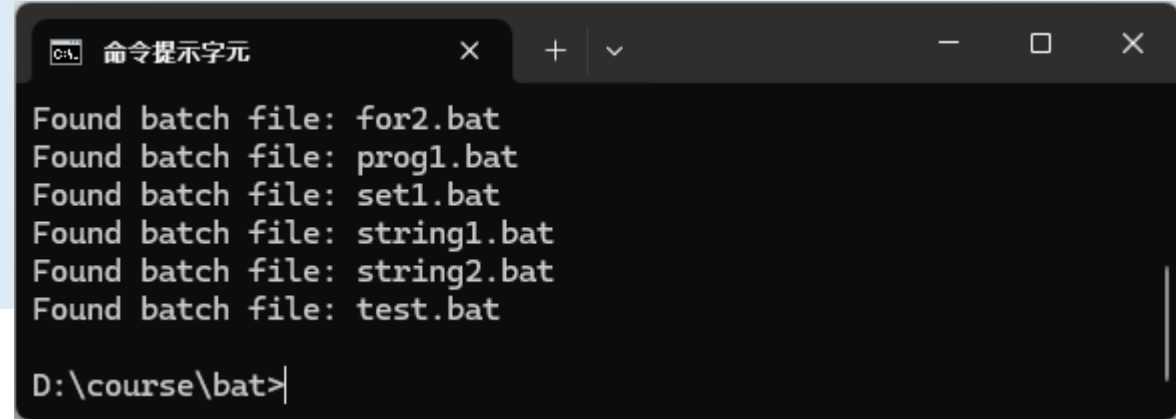


A screenshot of a Windows Command Prompt window titled "命令提示字元". The window shows the execution of a batch file named "list1" in the directory "D:\course\bat". The output of the batch file is displayed as follows:

```
D:\course\bat>list1
1
2
3
4
there
are
3
books
D:\course\bat>
```


for Iterating over files in a directory

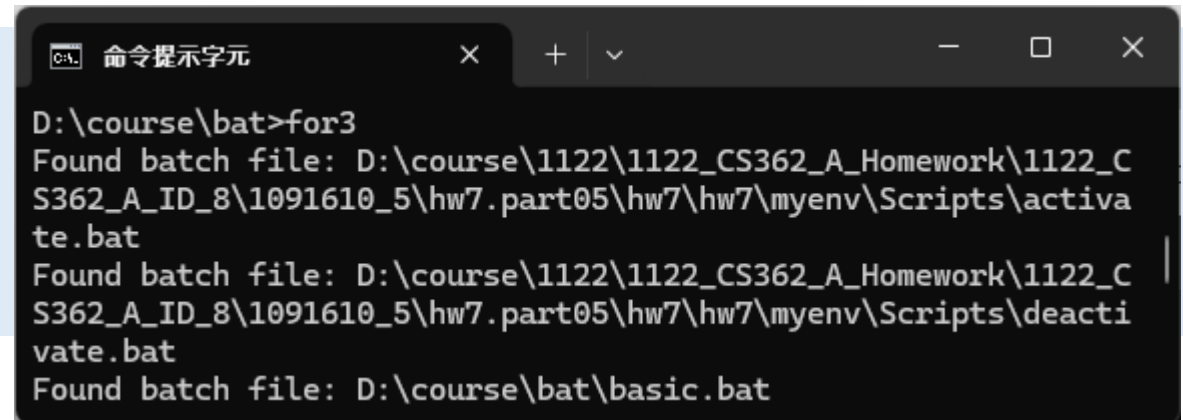
```
@echo off
for %%f in (*.bat) do (
    echo Found batch file: %%f
)
```

A screenshot of a Windows Command Prompt window titled "命令提示字元". The window shows the output of a batch script. It lists six batch files found in the current directory: for2.bat, prog1.bat, set1.bat, string1.bat, string2.bat, and test.bat. The prompt at the bottom is "D:\course\bat>".

```
命令提示字元
Found batch file: for2.bat
Found batch file: prog1.bat
Found batch file: set1.bat
Found batch file: string1.bat
Found batch file: string2.bat
Found batch file: test.bat
D:\course\bat>
```

recursively (/R)

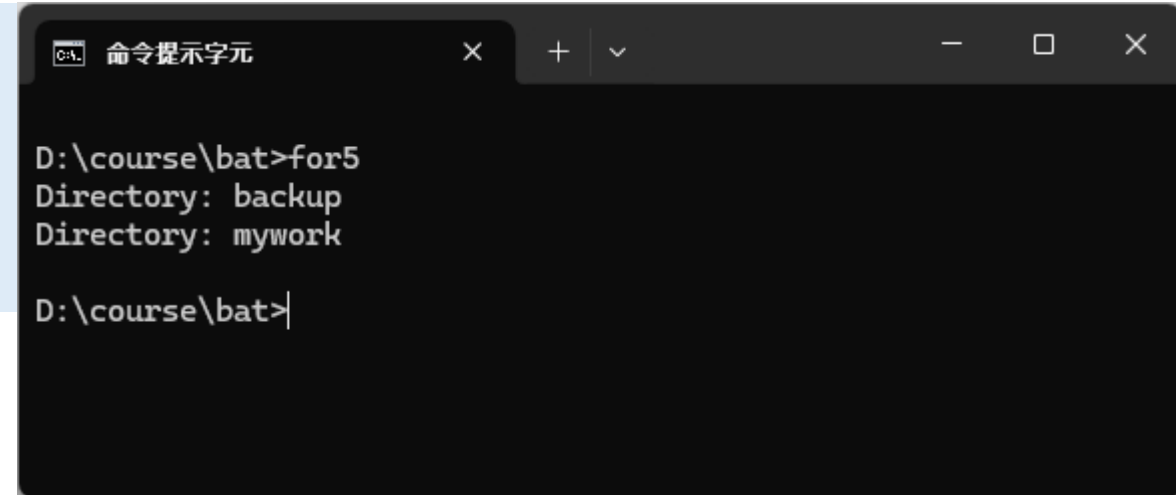
```
@echo off
for /R "D:\course" %%f in (*.bat) do (
    echo Found batch file: %%f
)
```

A screenshot of a Windows Command Prompt window titled "命令提示字元". The window shows the output of a batch script that recursively searches for batch files. It lists three batch files found: activate.bat, deactivate.bat, and basic.bat, along with their full paths. The prompt at the bottom is "D:\course\bat>for3".

```
命令提示字元
D:\course\bat>for3
Found batch file: D:\course\1122\1122_CS362_A_Homework\1122_CS362_A_ID_8\1091610_5\hw7.part05\hw7\hw7\myenv\Scripts\activate.bat
Found batch file: D:\course\1122\1122_CS362_A_Homework\1122_CS362_A_ID_8\1091610_5\hw7.part05\hw7\hw7\myenv\Scripts\deactivate.bat
Found batch file: D:\course\bat\basic.bat
```

for Iterating over files with wildcards * (/D for Directories)

```
@echo off
for /D %%d in (*) do (
    echo Directory: %%d
)
```



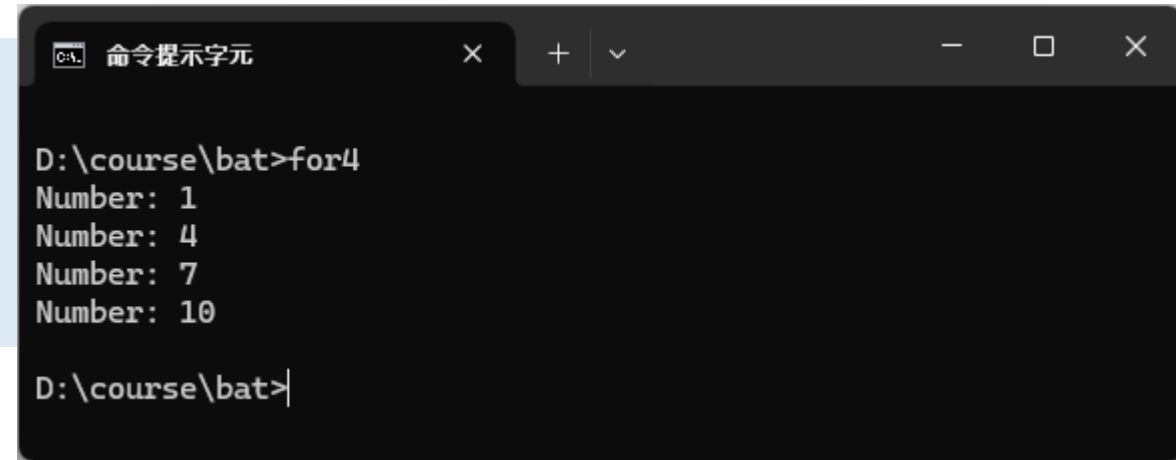
A screenshot of a Windows Command Prompt window titled "命令提示字元". The window shows the execution of a batch script. The prompt is "D:\course\bat>for5". The output shows "Directory: backup" and "Directory: mywork". The prompt then changes to "D:\course\bat>".

```
D:\course\bat>for5
Directory: backup
Directory: mywork

D:\course\bat>
```

for /L for Number Ranges

```
@echo off
for /L %%i in (1,3,10) do (
    echo Number: %%i
)
```



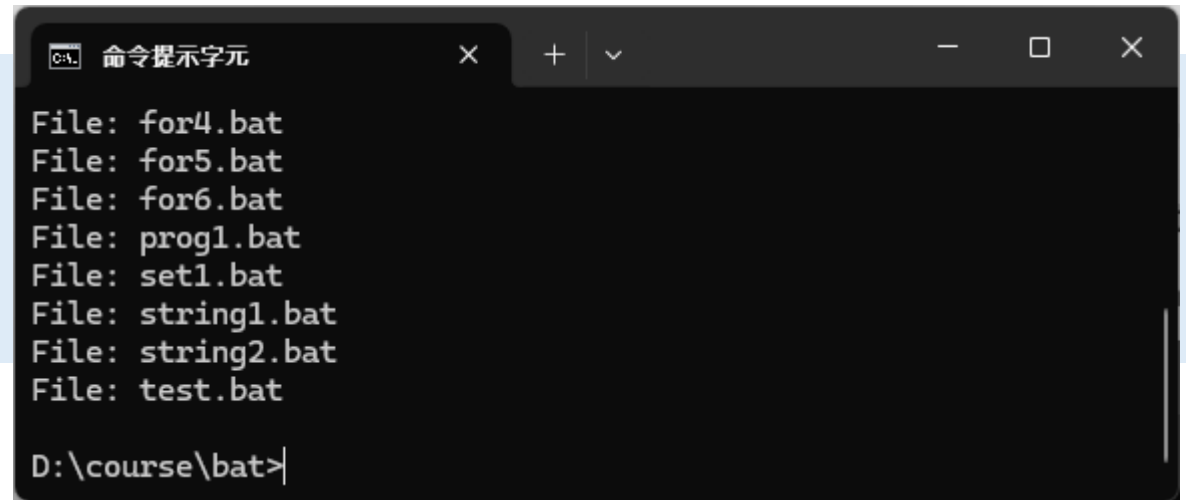
A screenshot of a Windows Command Prompt window titled "命令提示字元". The window shows the execution of a batch script. The command entered is "D:\course\bat>for4". The output of the script is displayed as follows:

```
D:\course\bat>for4
Number: 1
Number: 4
Number: 7
Number: 10

D:\course\bat>|
```

for /F to process the output of a command

```
@echo off
for /F "delims=" %%a in ('dir /b *.bat') do (
    echo File: %%a
)
```

A screenshot of a Windows Command Prompt window titled "命令提示字元". The window shows the output of a batch script that lists files in the current directory. The output is as follows:

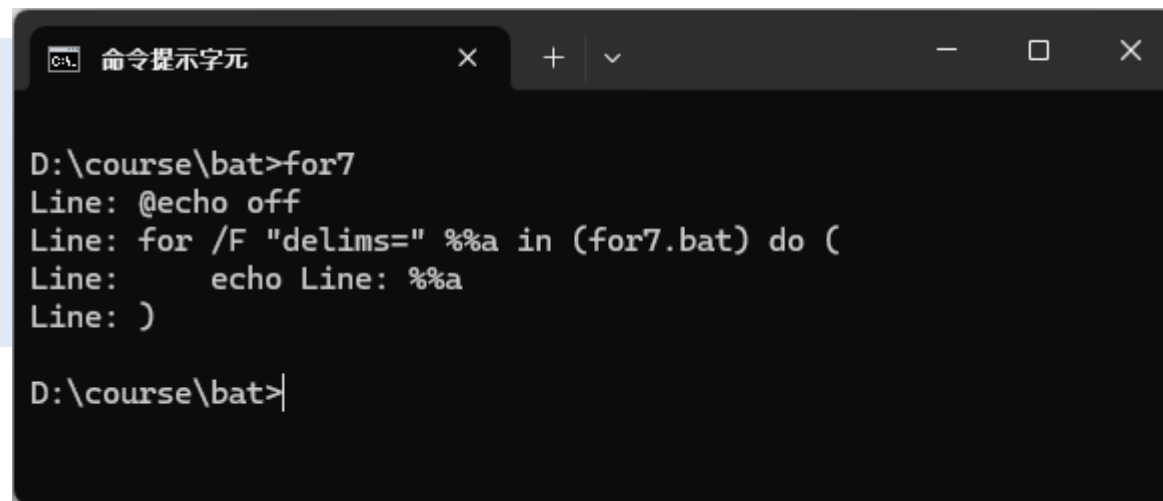
```
File: for4.bat
File: for5.bat
File: for6.bat
File: prog1.bat
File: set1.bat
File: string1.bat
File: string2.bat
File: test.bat
```

The prompt at the bottom is "D:\course\bat>".

```
命令提示字元
File: for4.bat
File: for5.bat
File: for6.bat
File: prog1.bat
File: set1.bat
File: string1.bat
File: string2.bat
File: test.bat
D:\course\bat>
```

for /F processing files line by line

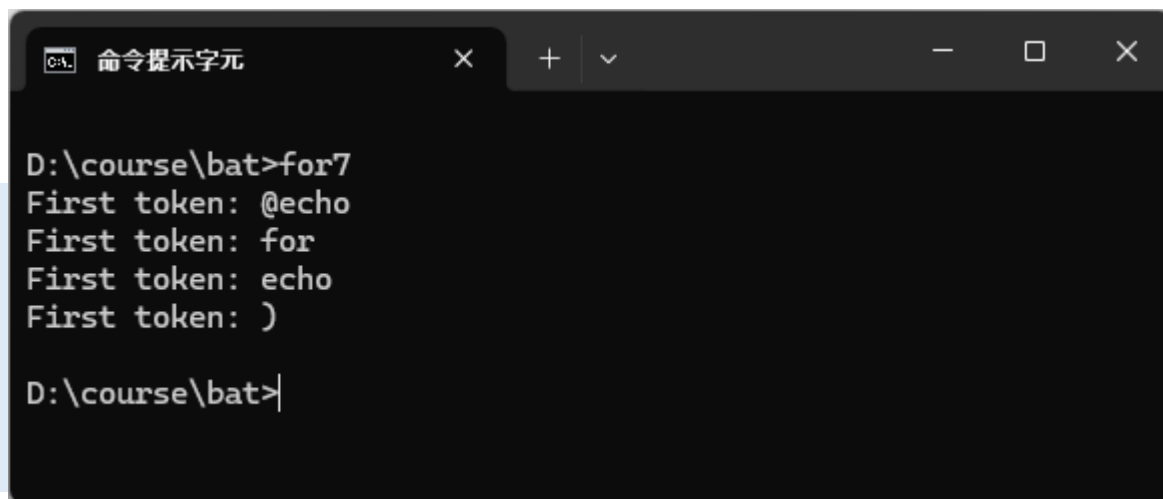
```
@echo off
for /F "delims=" %%a in (for7.bat) do (
    echo Line: %%a
)
```



```
命令提示字元
D:\course\bat>for7
Line: @echo off
Line: for /F "delims=" %%a in (for7.bat) do (
Line:     echo Line: %%a
Line: )

D:\course\bat>
```

```
@echo off
for /F "delims= " %%a in (for7.bat) do (
    echo Line: %%a
)
```



```
命令提示字元
D:\course\bat>for7
First token: @echo
First token: for
First token: echo
First token: )

D:\course\bat>
```

Arithmetic Operators

Operator	Description	Example
+	Addition of two operands	1 + 2 will give 3
-	Subtracts second operand from the first	2 - 1 will give 1
*	Multiplication of both operands	2 * 2 will give 4
/	Division of the numerator by the denominator	3 / 2 will give 1.5
%	Modulus operator and remainder of after an integer/float division	3 % 2 will give 1

Relational Operators

Operator	Description	Example
EQU	Tests the equality between two objects	2 EQU 2 will give true
NEQ	Tests the difference between two objects	3 NEQ 2 will give true
LSS	Checks to see if the left object is less than the right operand	2 LSS 3 will give true
LEQ	Checks to see if the left object is less than or equal to the right operand	2 LEQ 3 will give true
GTR	Checks to see if the left object is greater than the right operand	3 GTR 2 will give true
GEQ	Checks to see if the left object is greater than or equal to the right operand	3 GEQ 2 will give true

Logical Operators

Operator	Description
AND	This is the logical “and” operator
OR	This is the logical “or” operator
NOT	This is the logical “not” operator

Bitwise Operators

Operator	Description
&	This is the bitwise “and” operator
	This is the bitwise “or” operator
^	This is the bitwise “xor” or Exclusive or operator

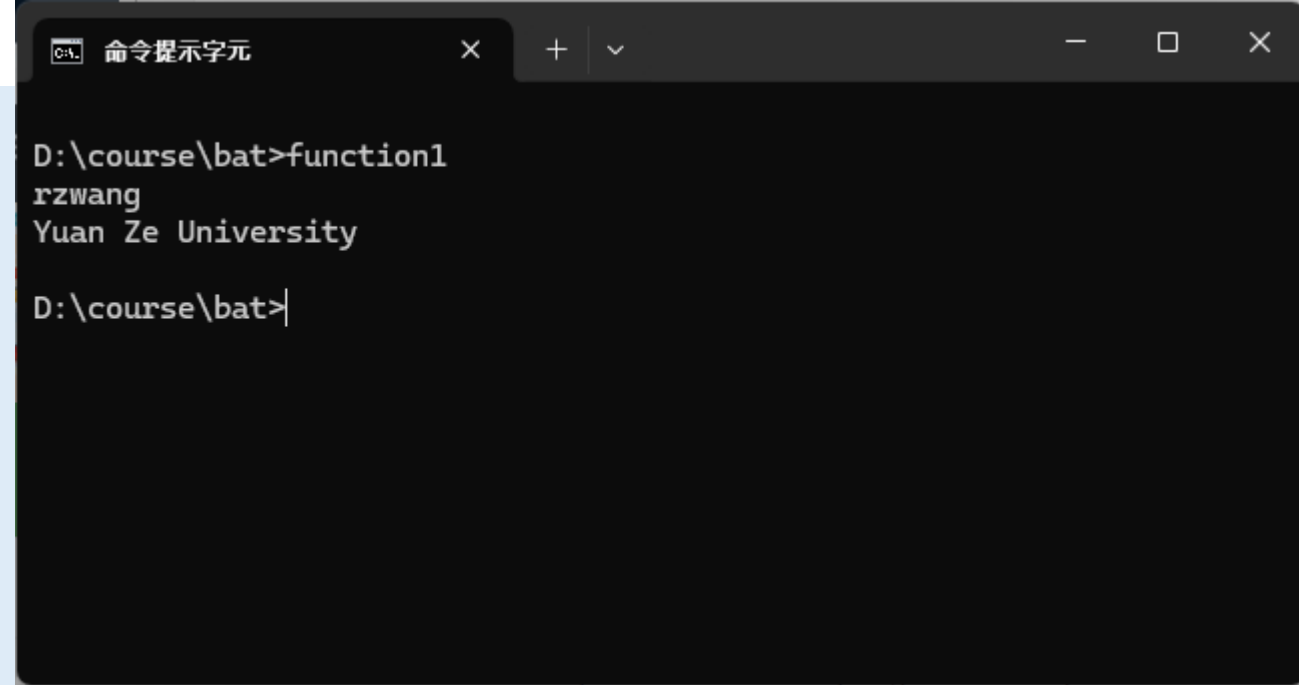
Assignment Operators

Operator	Description	Example
<code>+=</code>	This adds right operand to the left operand and assigns the result to left operand	Set /A a = 5 a += 3 Output will be 8
<code>-=</code>	This subtracts the right operand from the left operand and assigns the result to the left operand	Set /A a = 5 a -= 3 Output will be 2
<code>*=</code>	This multiplies the right operand with the left operand and assigns the result to the left operand	Set /A a = 5 a *= 3 Output will be 15
<code>/=</code>	This divides the left operand with the right operand and assigns the result to the left operand	Set /A a = 6 a/ = 3 Output will be 2
<code>%=</code>	This takes modulus using two operands and assigns the result to the left operand	Set /A a = 5 a% = 3 Output will be 2

function

```
@echo off
setlocal
call :printdata
exit /b %errorlevel%

:printdata
echo rzwang
echo Yuan Ze University
exit /b 0
```



```
命令提示字元
D:\course\bat>function1
rzwang
Yuan Ze University
D:\course\bat>
```

A function is a set of statements organized together to perform a specific task.

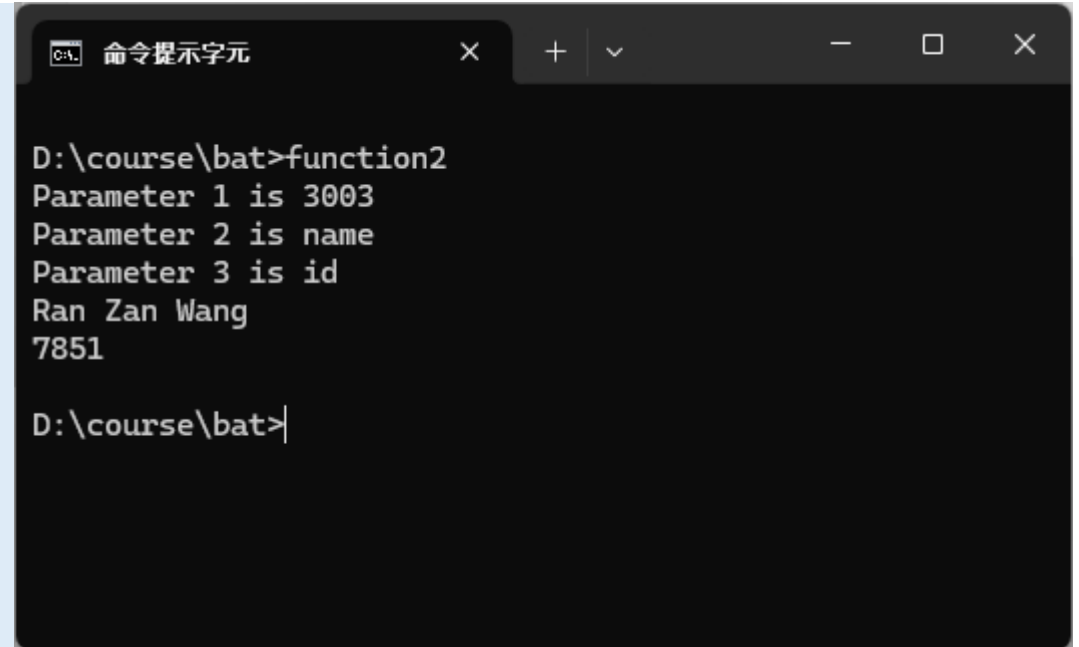
- **Function Declaration** – It tells the compiler about a function's name, return type, and parameters.
 - In Batch Script, a function is defined by using the label statement
- **Function Definition** – It provides the actual body of the function.
 - A function is called in Batch Script by using the call command. Following is the syntax.

`call :function_name`

function parameters and return values

```
@echo off
setlocal
call :setdata 3003 name id
echo %name%
echo %id%
exit /b %ERRORLEVEL%
```

```
:setdata
echo Parameter 1 is %~1
echo Parameter 2 is %~2
echo Parameter 3 is %~3
set "%~2=Ran Zan Wang"
set "%~3=7851"
exit /b 0
```

A screenshot of a Windows Command Prompt window titled "命令提示符" (Command Prompt). The window shows the execution of a batch file function. The prompt is "D:\course\bat>function2". The output is: "Parameter 1 is 3003", "Parameter 2 is name", "Parameter 3 is id", "Ran Zan Wang", and "7851". The prompt then returns to "D:\course\bat>".

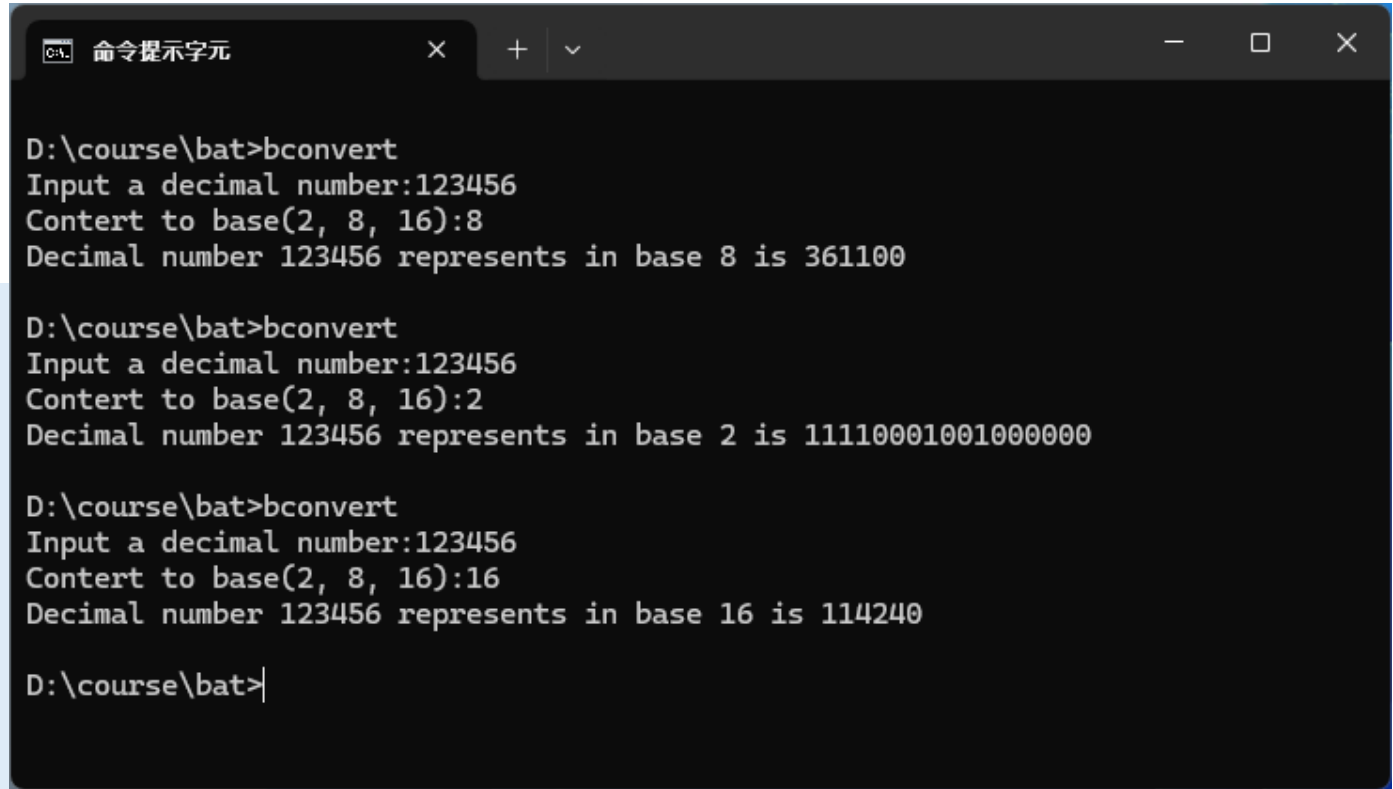
```
命令提示符
D:\course\bat>function2
Parameter 1 is 3003
Parameter 2 is name
Parameter 3 is id
Ran Zan Wang
7851
D:\course\bat>
```

Base conversion

```
@echo off
set /p inN=Input a decimal number:
set /p base=Contert to base(2, 8, 16):
```

```
set outN=
set /a num=%inN%
```

```
:loop1
set /a res=%num% %% %base%
set /a num=%num% / %base%
set outN=%res%%outN%
if %num% gtr 0 (
goto loop1
)
echo Decimal number %inN% represents in base %base% is %outN%
```



```
D:\course\bat>bconvert
Input a decimal number:123456
Contert to base(2, 8, 16):8
Decimal number 123456 represents in base 8 is 361100

D:\course\bat>bconvert
Input a decimal number:123456
Contert to base(2, 8, 16):2
Decimal number 123456 represents in base 2 is 11110001001000000

D:\course\bat>bconvert
Input a decimal number:123456
Contert to base(2, 8, 16):16
Decimal number 123456 represents in base 16 is 114240

D:\course\bat>
```

Create Folders

user.txt

```
1001001 Smith Wang
1001003 Bell Lee
1001005 Allen Huang
1001121 Kyle Lee
1003456 Pokemon Wang
1225678 Goodman Miller
```

```
命令提示字元
D:\course\bat\newFolder\1001001>cd ..
D:\course\bat\newFolder>dir
磁碟區 D 中的磁碟是 新增磁碟區
磁碟區序號: 6441-98A0

D:\course\bat\newFolder 的目錄

2024/10/16 下午 05:31 <DIR> .
2024/10/16 下午 05:31 <DIR> ..
2024/10/16 下午 05:31 <DIR> 1001001
2024/10/16 下午 05:31 <DIR> 1001003
2024/10/16 下午 05:31 <DIR> 1001005
2024/10/16 下午 05:31 <DIR> 1001121
2024/10/16 下午 05:31 <DIR> 1003456
2024/10/16 下午 05:31 <DIR> 1225678
            0 個檔案            0 位元組
            8 個目錄 373,840,556,032 位元組可用

D:\course\bat\newFolder>
```

```
@echo off
set inputFile=user.txt
set baseDir=newFolder
```

```
REM Create the base directory if it doesn't exist
if not exist %baseDir% (
    mkdir %baseDir%
)
```

```
REM Loop through each line in the file
for /f "tokens=1,2,3 delims= " %%A in (%inputFile%) do (
    REM Check if the folder for the ID already exists
    if not exist %baseDir%\%%A (
        mkdir %baseDir%\%%A
    ) else (
        echo Folder %%A already exists.
    )
```

```
cd %baseDir%\%%A
```

```
REM Check if the batch file for the user already exists
if not exist %%B%%C.bat (
    echo A new file > %%B%%C.bat
) else (
    echo File %%B%%C.bat already exists.
)
cd ..\..
)
```

```
echo Folders and files created successfully.
```

Add record to file

```
@echo off
setlocal enabledelayedexpansion

set /p name="Enter Student Name:"
set /p email="Enter Student Email:"
set /p id="Enter Student ID:"

:: Create a temporary file
set tempfile="temp.xml"
set originalfile="cs140a.xml"

:: Delete temp file if it exists
if exist %tempfile% del %tempfile%
```

```
:: Read the original file and write everything except the closing
</Students> tag
for /f "usebackq delims=" %%a in (%originalfile%) do (
    set line=%%a
    set myVar=!line: =!
    :: add Student informaion before </Students> tag
    if "!myVar!" == "</Students>" (
        echo ^<Student^> >> %tempfile%
        echo ^<Name^>%name%^</Name^> >> %tempfile%
        echo ^<Email^>%email%^</Email^> >> %tempfile%
        echo ^<Student ID^>%id%^</Student ID^> >> %tempfile%
        echo ^</Student^> >> %tempfile%
    )
    echo !line! >> %tempfile%
    if "!myVar!"=="</CourseInfo>" echo. >> %tempfile%
    if "!myVar!"=="</Teacher>" echo. >> %tempfile%
    if "!myVar!"=="</TeachingAssistants>" echo. >> %tempfile%
)
:: Overwrite the original file with the updated temp file
move /Y %tempfile% %originalfile%

echo Student %name% added successfully to the Students section!
```

```
<?xml version="1.0" encoding="UTF-8"?>
<Class>

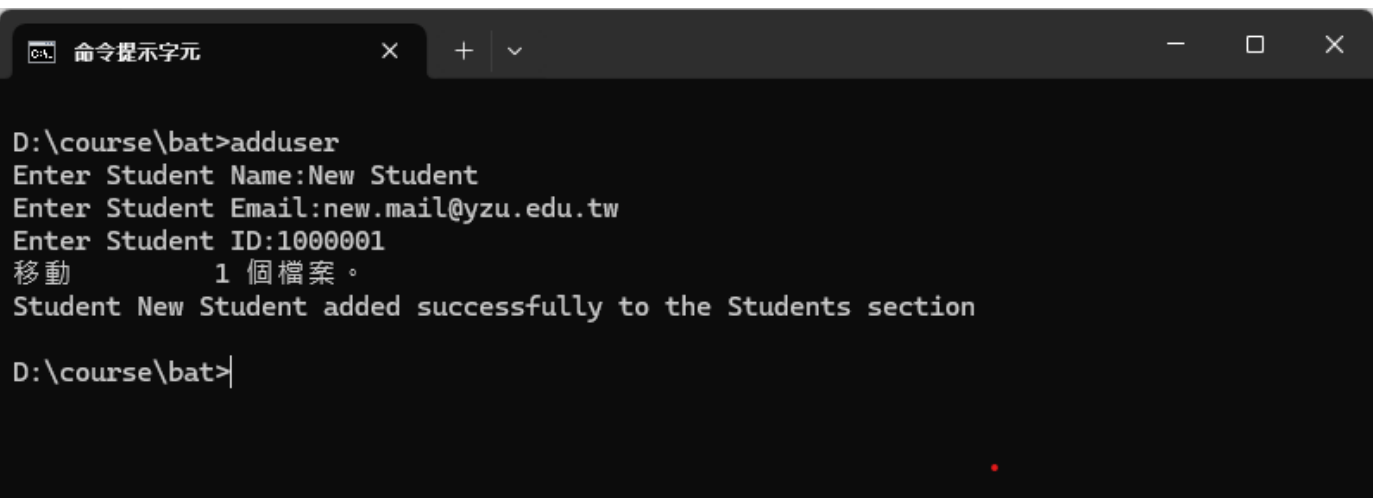
<CourseInfo>
<CourseName>Introduction to Computer Science</CourseName>
<CourseCode>CS140A</CourseCode>
<Semester>Fall 2024</Semester>
<Credits>3</Credits>
<Classroom>Room 60104</Classroom>
<Schedule>
<Day>Thrsday</Day>
<Time>10:10 AM - 12:00 AM</Time>
</Schedule>
<Schedule>
<Day>Friday</Day>
<Time>09:00 AM - 10:00 AM</Time>
</Schedule>
</CourseInfo>

<Teacher>
<Name>Ran-Zan Wang</Name>
<Email>rzwang@saturn.yzu.edu.tw</Email>
<Phone>03-4638800-3003</Phone>
<Office>Room 61014</Office>
</Teacher>

<TeachingAssistants>
<TA>
<Name>Zhi-Jie Wu</Name>
<Email>s1126028@mail.yzu.edu.tw</Email>
<Phone>03-4638800-2367-4112</Phone>
</TA>
<TA>
<Name>Zheng-Yang Weng</Name>
<Email>s1136012@mail.yzu.edu.tw</Email>
<Phone>03-4638800-2367-4112</Phone>
</TA>
</TeachingAssistants>

<Students>
<Student>
<Name>Student 1</Name>
<Email>s0000001@mail.yzu.edu.tw</Email>
<Student ID>0000001</Student ID>
</Student>
<Student>
<Name>Student 2</Name>
<Email>s0000002@mail.yzu.edu.tw</Email>
<Student ID>0000002</Student ID>
</Student>
</Students>

</Class>
```



```
D:\course\bat>adduser
Enter Student Name:New Student
Enter Student Email:new.mail@yzu.edu.tw
Enter Student ID:1000001
移動      1 個檔案。
Student New Student added successfully to the Students section

D:\course\bat>
```

```
<Students>
<Student>
<Name>Student 1</Name>
<Email>s0000001@mail.yzu.edu.tw</Email>
<Student ID>0000001</Student ID>
</Student>
<Student>
<Name>Student 2</Name>
<Email>s0000002@mail.yzu.edu.tw</Email>
<Student ID>0000002</Student ID>
</Student>
<Student>
<Name>New Student</Name>
<Email>new.mail@yzu.edu.tw</Email>
<Student ID>1000001</Student ID>
</Student>
</Students>
</Class>
```

References

- [MS-Dos Commands](#)
- [Batch File Script](#)